Using Templates

When you write a view, you're writing a template: something that will get expanded to generate the final result. To understand how these templates work, we need to look at three areas:

- Where the templates go
- The environment they run in
- What goes inside them

Where Templates Go

The `render` method expects to find templates in the `app/views` directory of the current application. Within this directory, the convention is to have a separate subdirectory for the views of each controller. Our Depot application, for instance, includes products and store controllers. As a result, our application has templates in `app/views/products` and `app/views/store`. Each directory typically contains templates named after the actions in the corresponding controller.

You can also have templates that aren't named after actions. You render such templates from the controller using calls such as these:

```
render(action:   'fake_action_name')
render(template: 'controller/name')
render(file:     'dir/template')
```

The last of these allows you to store templates anywhere on your filesystem. This is useful if you want to share templates across applications.

The Template Environment

Templates contain a mixture of fixed text and code. The code in the template adds dynamic content to the response. That code runs in an environment that gives it access to the information set up by the controller:

- All instance variables of the controller are also available in the template. This is how actions communicate data to the templates.